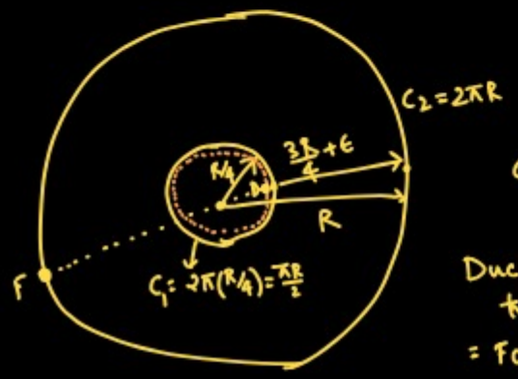
Fox's speed is 4 times that of duck's speed in water.

can duck escape the fox?

Circumference $= 2\pi R$



$C_2 = 4C_1$

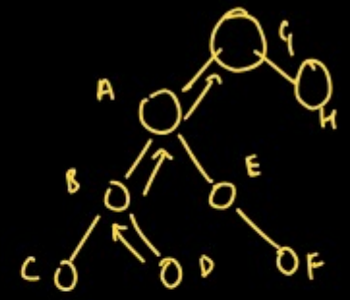
Duck's time to traverse C_2 = Fox's time to traverse C_1

Duck's distance $= \frac{3R}{4} + \epsilon = \frac{3.01}{4} R$ ($\epsilon \to \frac{0.01R}{4}$)

Fox's distance $= \pi R = 3.14R = 3.14R$

Heaps

- Max heaps, min heaps
- Max heaps:
  - Heap is a tree structure



Binary tree: each element has max 2 children

B is parent of C

C, D are children of B

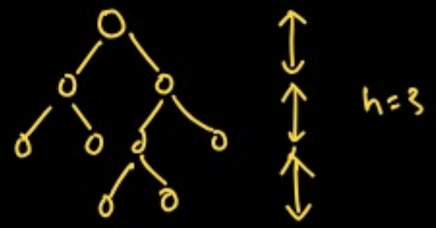
C is left child of B

D is right child of B

C, D, F, are leaves

A is an ancestor of C

G is the root of tree

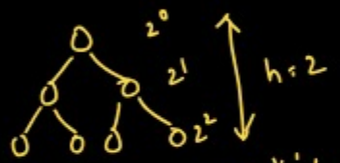


h = 0

h = 1

h = 4

perfect binary tree
all nodes except in the last level have exactly 2 children



perfect binary tree of 'h' height,
nodes $= 2^{h+1} - 1$

nodes: $2^0 + 2^1 + 2^2 + \dots + 2^h$

$= \frac{1(2^{h+1}-1)}{2-1} = 2^{h+1} - 1$

$a = 2^0 = 1$

$r = 2$

n = # terms $= h+1$

All complete BTs are perfect ✗

All perfect BTs are complete ✓

G.P.:

$\underbrace{a + ar + ar^2 + \dots + ar^{n-1}}_{\text{\# terms} = n} = \frac{a(r^n - 1)}{r-1}$

first term: a

r = common ratio

Complete binary tree

perfect ✗
complete ✗

Except leaves, all nodes have exactly 2 children

In perfect binary tree of 'n' nodes, what is its height?

- say height is h

$$n = 2^{h+1} - 1$$

$$\Rightarrow n+1 = 2^{h+1}$$

$$\Rightarrow \log_2(n+1) = \log_2(2^{h+1})$$

$$\Rightarrow \log_2(n+1) = (h+1)\log_2 2$$

$$= (h+1)(1)$$

$$= h+1$$

$$\Rightarrow \boxed{h = \log_2(n+1) - 1}$$

$$\Rightarrow \boxed{h = \Theta(\log n)}$$

$$16 = 2^x$$

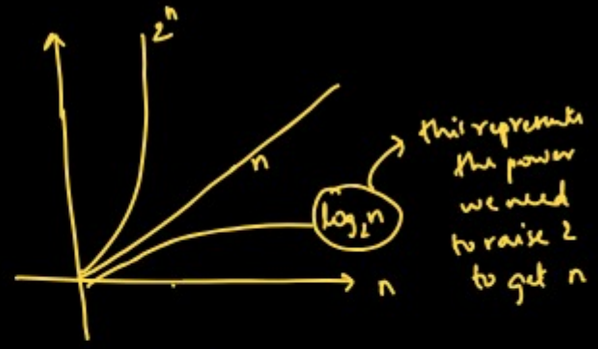
$$\log_2 16 = \log_2 2^x$$

$$4 = x \log_2 2$$

$$\boxed{4 = x(1)}$$

$$\log_2 16 = 4$$

$$16 \xrightarrow{2} 8 \xrightarrow{2} 4 \xrightarrow{2} 2 \xrightarrow{2} 1$$



$$\log_2 2 = 1$$

$$\log_2 4 = 2$$

$$\log_2 8 = 3$$

$$\log_2 1024 = 10$$

$$\log_2 1000000000 = 30$$

n is to 2^n

what $\log_2 n$ is to n is

$$= 2^{\log_2 n}$$

$$2^n >> n \qquad n << 2^n$$

$$n >> \log_2 n$$

$$\log_2 n << n$$

· Max-heaps

$1 < \log(\log n) < \log n < n < n^2 < \dots < 2^n$

heap {binary tree}

$n = 4096$ $\log_2 n = 12$ $\log(\log_2 n) = 3.$

$n = 1$ bil $\log_2 n = 30$ $\log(\log) = 5$

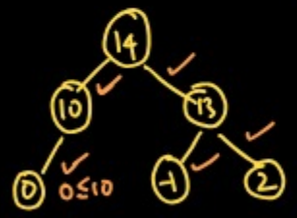
$n = 1$ trill $= 40$ $= \boxed{5.}$

$n =$

Every node's value $\geq$ the children's values

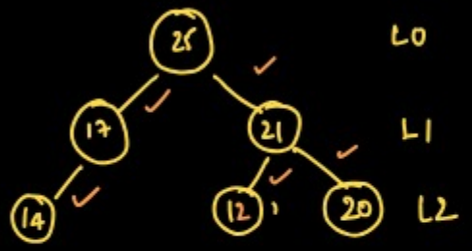
Every node's value $\leq$ its parent's value

Heap property



· In a max heap, the root's value will be the largest (max) value in the whole tree.

The min value has to be in one of the leaves

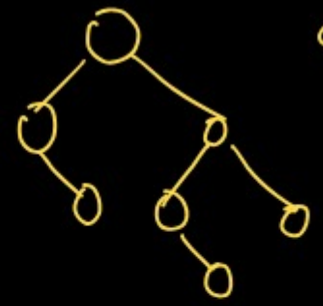


Binary tree ✓

Heap ✗

BST ✗

is a valid max heap but $h = \Theta(n)$

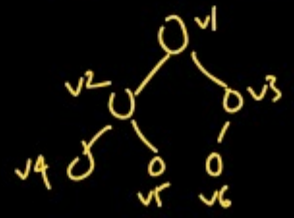


```
class HeapNode {
    int value;
    HeapNode left;
    HeapNode right;
    HeapNode parent
}
```

32 byte structure

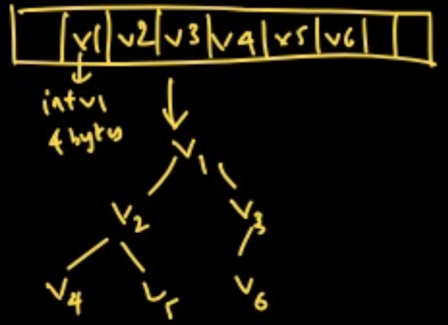
Total memory = 32n bytes

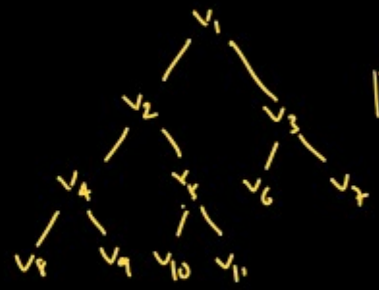
· We maintain each binary heap only as a complete binary tree



$\boxed{h = \Theta(\log n)}$

Total memory = 4n bytes





0	1	2	3	4	5	6	7	8	9	10	11
-	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}	v_{11}

v_2's parent : v_1

v_6's parent : v_3

$\text{parent}(i) = \lfloor i/2 \rfloor$

$\text{left}(i) = 2i$

$\text{right}(i) = 2i+1$

Min Heap

	getMin()	insert(ele)	removeMin()
unsorted array	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
sorted array	$\Theta(1)$	$O(n)$	$\Theta(n)$
sorted linked list	$\Theta(1)$	$O(n)$	$\Theta(1)$
heaps	$\Theta(1)$	$O(\log n)$	$O(\log n)$

finding the insertion position takes $O(n)$ time then insertion itself is $\Theta(1)$

finding the insertion index is $O(\log n)$ using Binary Search But actually inserting takes $O(n)$ because of need to shift elements.

$n = 4 \Rightarrow h = 2$

$\vdots$

$n = 7 \Rightarrow h = 2$

2^k nodes $\downarrow$ $2^{k+1} - 1$ nodes $\Big\}$ $h = k$

$$\log_2 n = \frac{\log_3 n}{\log_3 2} = k_1 \log_3 n$$

$$\Theta(\log_2 n) = \Theta(\log_3 n) = \boxed{\Theta(\log n)}$$

L0 full ✓
L1 full ✓
L2 not full ✗

Perfect BT:

- when every node has exactly 2 children except the nodes in the <u>last level</u> (or) Every level is full

last level

↳ Every node in the last level is a leaf
But not all leaves are necessarily in the last level.

- Consider a tree where every node has exactly 2 children except the leaves

last level

Not a perfect binary tree

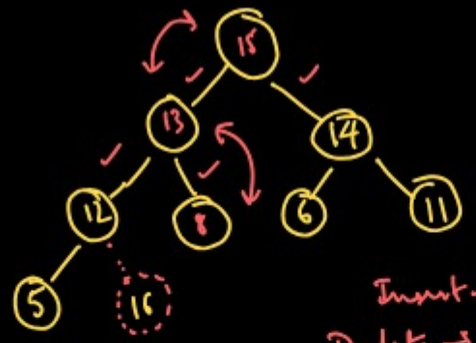
- complete binary tree
  1. All levels except the last level should be full
  2. The last level should be filled from left to right

L0 full ✓
L1 full ✓
Last level filled from L to R ✓

L0 ✓
L1 ✓
Last level filled from L to R ✓

- For a general binary tree
  $h = \Omega(\log n)$
  $h = O(n)$
- For a perfect binary tree
  $h = \Theta(\log n)$ → proved in frame 2
- For a complete binary tree
  $h = \Theta(\log n)$

- Storing heaps as complete binary trees
  Adv 1: the elements can be stored in an array without needing pointers to left child, right child, parent
  ⇒ 4n bytes vs 3n bytes memory
  Adv 2: $h = \Theta(\log n)$



Insert → $O(h)$ → $O(\log n)$
Delete → $O(h)$ → $O(\log n)$

- siftUp or swim
  - starting at bottom most level, fix heap property level by level going potentially until the root
- siftDown or sink
  - start at root & fix heap property level by level going potentially until the last level.

5 2 1 6 3 4

i min

1 2 5 6 3 4

i min

1 2 5 6 3 4

i min

1 2 3 6 5 4

i min

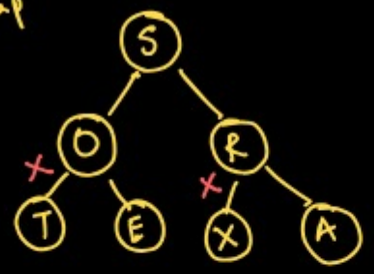
1 2 3 4 5 6

i min

1 2 3 4 5 6

i

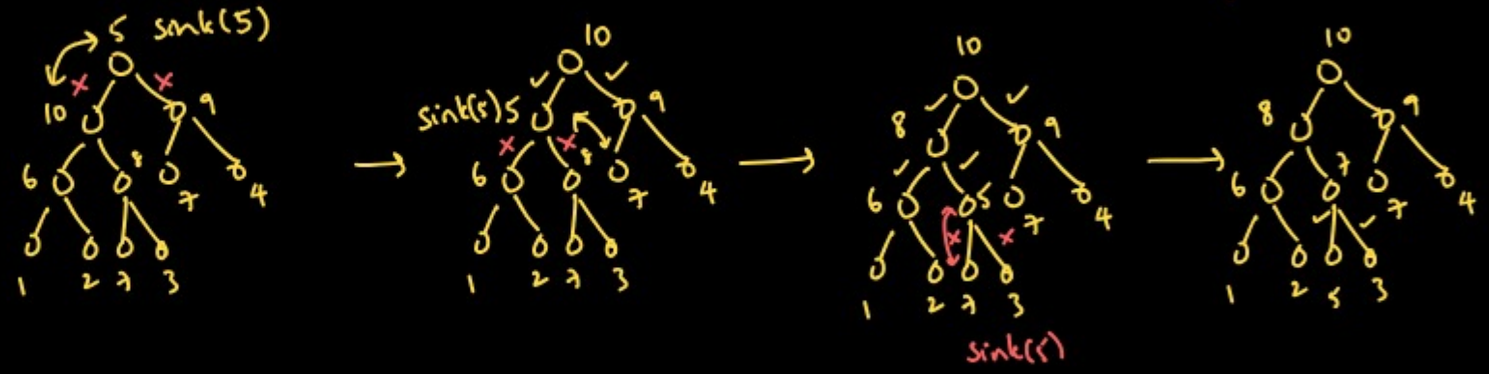
Max heap



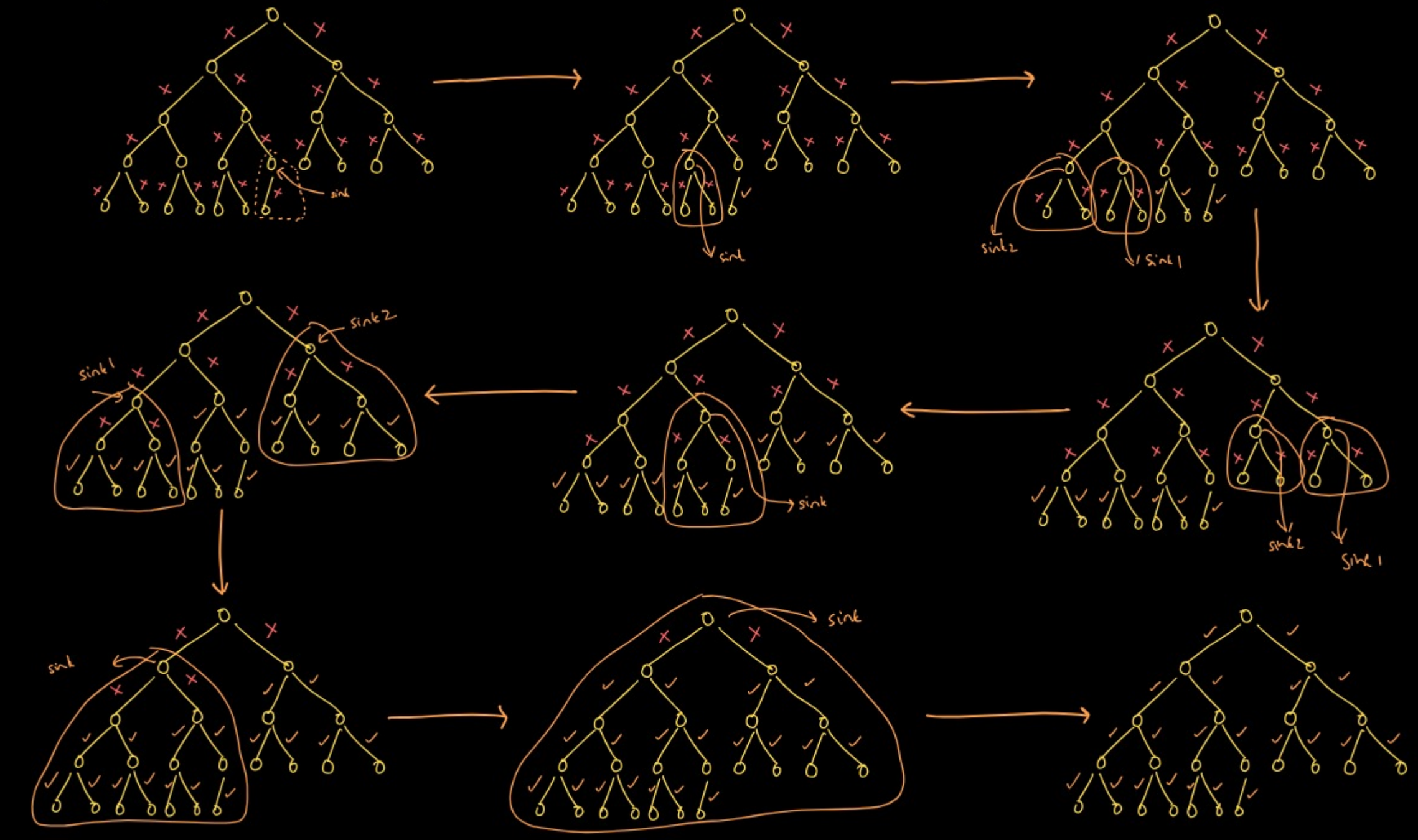
≡ | - | S | O | R | T | E | X | A |

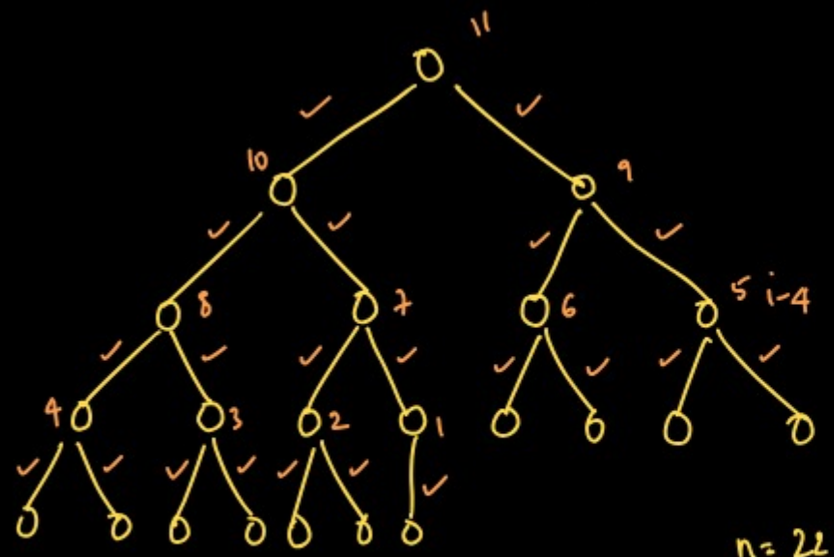
Build heap from the given array

sink (node) can fix the heap only if the heap property is violated between the node & its children & its not violated anywhere else.



Build a heap


sink
sink
sink2
sink1
sink2
sink1
sink
sink1
sink2
sink
sink
sink



leaves in a complete binary tree of n nodes $\sim n/2$

non-leaf nodes in a CBT $\sim n/2$

$n = 22 \Rightarrow \#l = 11$
$n = 23 \Rightarrow \#l = 12$
$n = 24 \Rightarrow \#l = 12$
$n = 25 \Rightarrow \#l = 13$
$n = 26 \Rightarrow \#l = 13$

Build heap running time:

- We are calling sink on all non-leaves
  $\Rightarrow$ #sink ops $= \sim \frac{n}{2}$
- # ops in each sink $\leq h$ or $\log n$
- # total ops in build heap $\leq \frac{n}{2} \times \log n$

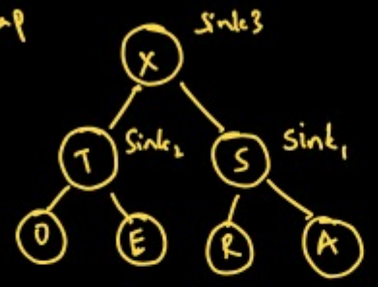
$O(n \log n)$

level	#nodes	#Compares

Heap sort

- Build heap – $O(n)$

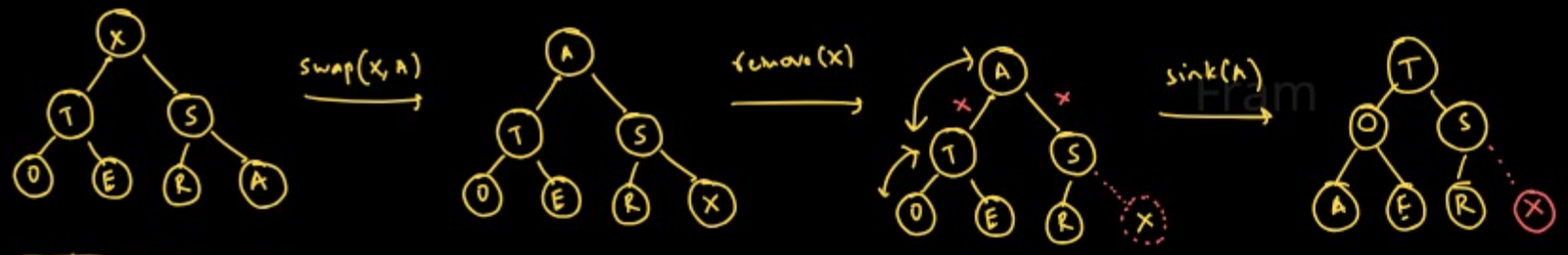
Max heap



$\equiv$ | - | X | T | S | O | E | R | A |

- Do the following steps until no more elements in heap
  1. exchange root with last element – $\Theta(1)$
  2. remove last element from heap – $\Theta(1)$
  3. sink(root) – $O(\log n)$

$\Big\}\ O(\log n)$

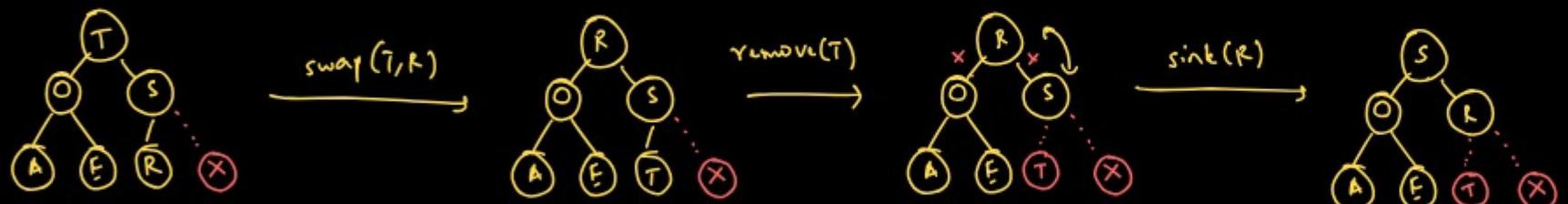


-	X	T	S	O	E	R	A

-	A	T	S	O	E	R	X

-	A	T	S	O	E	R	X

-	T	O	S	A	E	R	X



-	T	O	S	A	E	R	X

-	R	O	S	A	E	T	X

-	R	O	S	A	E	T	X

-	S	O	R	A	E	T	X

⋮

Heap sort → first algo that is guaranteed $n \log n$ time in worst case & is inplace

- RT - $O(n \log n)$
- Inplace ✓
- Stable?
  - Long exchange distances? Yes ⇒ Not stable

Merge sort

- $\Theta(n \log n)$
- Not inplace X
- Stable ✓

Quick sort

- $n \log n$ time on average
  n^2 time in worst case
- Inplace ✓
- Stable X

Geometric progression (GP):

- $a, ar, ar^2, ar^3, \dots$
  $T_1 \quad T_2 \quad T_3 \quad T_4$

$$T_n = ar^{n-1}$$

sum of first 'n' terms

$$S_n = a + ar + ar^2 + \dots + ar^{n-1}$$
$$= \frac{a(r^n - 1)}{(r-1)} \text{ or } \frac{a(1-r^n)}{(1-r)}$$

$$S_n = a + ar + ar^2 + \dots + ar^{n-1}$$
$$rS_n = \quad ar + ar^2 + \dots + ar^{n-1} + ar^n$$
$$S_n - rS_n = a + 0 + 0 + \dots + 0 - ar^n$$
$$\Rightarrow S_n(1-r) = a(1-r^n)$$
$$\Rightarrow \boxed{S_n = \frac{a(1-r^n)}{1-r}} \rightarrow \text{closed form}$$

If $0 < r < 1$ then $a + ar + ar^2 + ar^3 + \dots \infty = \frac{a}{1-r}$

Taken Bucket of 2 ltrs

- Each step, fill half of remaining capacity

Step 1 : 1 ltr
Step 2 : $1/2$ ltr
Step 3 : $1/4$ ltr
$\vdots$
2 ltrs

$$1 + 1/2 + 1/2^2 + \dots \infty \rightarrow 2$$

- $r > 1$

$$S_n = a + ar + ar^2 + \dots + ar^{n-1} = \frac{a(r^n - 1)}{(r-1)}$$

$$S_\infty = \lim_{n \to \infty} \frac{a(r^n - 1)}{(r-1)}$$

As $n \to \infty$, $r^n \to \infty$ if $r > 1$

$$S_\infty \to \infty$$

- $0 < r < 1$

$$S_n = a + ar + ar^2 + \dots + ar^{n-1}$$
$$= \frac{a(1-r^n)}{(1-r)}$$

$$S_\infty = \lim_{n \to \infty} \frac{a(1-r^n)}{(1-r)}$$

$$S_\infty \to \frac{a(1-0)}{1-r}$$

$$\boxed{S_\infty \to \frac{a}{1-r}}$$

As $n \to \infty$, $r^n \to 0$ if $0 < r < 1$

$r = 0.99$

$r^{10} \sim 0.9$
$r^{20} \sim 0.81$
$r^{50} \sim 0.6$
$r^{100} \sim 0.36$
$r^{500} \sim 0.06$

1. $a + ar + ar^2 + \cdots + ar^{n-1} = \dfrac{a(r^n - 1)}{r-1}$

If $r = 1$,

$$\underbrace{a + a + a + \cdots + a}_{n \text{ terms}} = an$$

(2) If $|r| < 1$, $a + ar + ar^2 + \cdots \infty = \dfrac{a}{1-r}$

(3) If $a = 1$, $\boxed{1 + r + r^2 + \cdots \infty = \dfrac{1}{1-r}}$

(4)

$$S = t + 2t^2 + 3t^3 + 4t^4 + \cdots \infty \qquad (0 < t < 1)$$

$$tS = \qquad t^2 + 2t^3 + 3t^4 + \cdots \infty$$

$$S - tS = t + t^2 + t^3 + t^4 + \cdots \infty$$

$$= t\,(\underbrace{1 + t + t^2 + \cdots \infty})$$

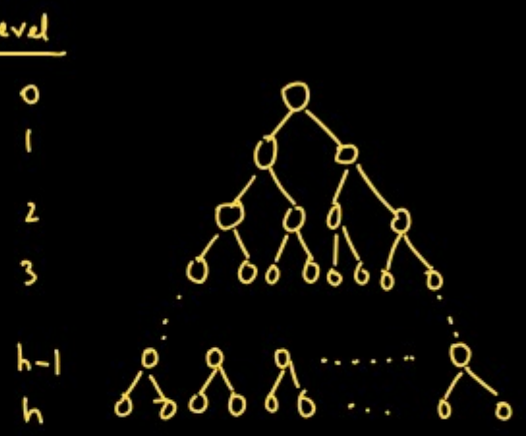
$$= t\left(\frac{1}{1-t}\right)$$

from (3)
$1 + r + r^2 + \cdots \infty = \dfrac{1}{1-r}$

$$S(1-t) = \frac{t}{1-t}$$

$$\boxed{S = \frac{t}{(1-t)^2}}$$

Build Heap



#nodes	#compares for sink on each node
$2^0 = 1$	$\le 2h$
$2^1 = 2$	$\le 2(h-1)$
$2^2 = 4$	$\le 2(h-2)$
$2^3 = 8$	$\le 2(h-3)$
$\vdots$	$\vdots$
2^{h-1}	≤ 2
2^h	0

Total #compares for build heap $= (2^0 \times 2h) + (2^1 \times 2(h-1)) + (2^2 \times 2(h-2)) + (2^3 \times 2(h-3)) + \cdots + (2^{h-1} \times 2) + 0$

$h = \log_2(n+1) - 1$

$\Rightarrow h + 1 = \log_2(n+1)$

$\Rightarrow 2^{h+1} = 2^{\log_2(n+1)}$

$\Rightarrow 2^{h+1} = n+1$

$\Rightarrow 2^h \times 2 = n+1$

$\Rightarrow 2^h = \dfrac{n+1}{2}$

$\Rightarrow \dfrac{2^h}{2^k} = \dfrac{n+1}{2^{k+1}}$

$\Rightarrow \boxed{2^{h-k} = \dfrac{n+1}{2^{k+1}}}$

$k = \log_2 a \rightarrow$ # times we have to divide a by 2 to get to 1

$$\begin{array}{ll} a & \times 2 \\ \vdots & \\ & \times 2 \\ & \times 2 \\ 1 & \end{array} \qquad \begin{array}{ll} a & \\ a/2 & /2 \\ a/4 & /2 \\ \vdots & /2 \\ 1 & \end{array}$$

$2^k = a$

$\boxed{2^{\log_2 a} = a}$

$\boxed{b^{\log_b a} = a}$

$\boxed{a^m \cdot a^n = a^{m+n}}$

Total #compares for build heap

$$\le (2^0 \times 2h) + (2^1 \times 2(h-1)) + (2^2 \times 2(h-2)) + (2^3 \times 2(h-3)) + \cdots + (2^{h-1} \times 2) + 0$$

$$\le (2^{h-1} \times 2) + (2^{h-2} \times 2(2)) + (2^{h-3} \times 2(3)) + \cdots + (2^3 \times 2(h-3)) + (2^2 \times 2(h-2)) + (2^1 \times 2(h-1)) + (2^0 \times 2(h-0))$$

$$\le \left(\frac{n+1}{2^{1+1}} \times 2(1)\right) + \left(\frac{n+1}{2^{2+1}} \times 2(2)\right) + \left(\frac{n+1}{2^{3+1}} \times 2(3)\right) + \cdots + \left[\frac{n+1}{2^{(h-3)+1}} 2(h-3)\right] + \left[\frac{n+1}{2^{(h-2)+1}} \times 2(h-2)\right] + \left[\frac{n+1}{2^{(h-1)+1}} \times 2(h-1)\right] + \left[\frac{n+1}{2^{(h-0)+1}} \times 2(h-0)\right]$$

$$\le \frac{n+1}{2^1} + \frac{(n+1)2}{2^2} + \frac{(n+1)3}{2^3} + \cdots + \frac{n+1(h-2)}{2^{h-2}} + \frac{(n+1)(h-1)}{2^{h-1}} + \frac{(n+1)h}{2^h}$$

$$\frac{(n+1)2(h-2)}{2^{(h-2)+1}} = \frac{(n+1)\cancel{2}(h-2)}{2^{h-2} \times \cancel{2}} = \frac{(n+1)(h-2)}{2^{h-2}}$$

$$\le (n+1)\left[\left(\tfrac{1}{2}\right)^1 + 2\left(\tfrac{1}{2}\right)^2 + 3\left(\tfrac{1}{2}\right)^3 + \cdots + (h-2)\left(\tfrac{1}{2}\right)^{h-2} + (h-1)\left(\tfrac{1}{2}\right)^{h-1} + h\left(\tfrac{1}{2}\right)^h\right]$$

$$\le (n+1)\left[\left(\tfrac{1}{2}\right)^1 + 2\left(\tfrac{1}{2}\right)^2 + 3\left(\tfrac{1}{2}\right)^3 + \cdots \infty\right]$$

$$t + 2t^2 + 3t^3 + \cdots \infty = \frac{t}{(1-t)^2}$$

$\{ 0 < t < 1$

where $t = \frac{1}{2}$

$$\le (n+1) \times \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2}$$

$$\le (n+1) \times \frac{4}{2}$$

$$\boxed{\#\text{compares} \le 2(n+1)}$$

$$\#\text{exchanges} \sim \frac{\#\text{compares}}{2} = (n+1)$$

Total work for build heap: $\#\text{compares} + \#\text{exchanges} \le 3(n+1) = O(n)$

Problem: Given an array of 'n' integers,
Sort the 'k' largest elements & put them in the last 'k' slots of the given array

4, 3, -1, 6, 2, 0, 8, 5 $\longrightarrow$ $\underbrace{5, 6, 8}_{\text{sorted}}$

$n = 8$
$k = 3$

(k bubble sort's passes also works)

Approach 1: Run selection sort's inner loop 'k' times

4, 3, -1, 6, 2, 0, 8, 5
i, max

$\boxed{8}$ 3 -1 6 2 0 4 5
i, max

$\boxed{8\ 6}$ -1 3 2 0 4 5
i, max

$\boxed{8\ 6\ 5}$ 3 2 0 4 -1

4, 3, -1, 6, 2, 0, 8, 5
max, i

4 3 -1 6 2 0 5 $\boxed{8}$
max, i

4 3 -1 5 2 0 $\boxed{6\ 8}$
max, i

4 3 -1 0 2 $\boxed{5\ 6\ 8}$

RT: $\underbrace{(n) + (n-1) + (n-2) + \cdots}_{\text{'k' times}}$

$: \Theta(nk)$

Approach 2:

- Sort the whole array then the last 'k' elements would be the k largest elements in sorted order
- RT: $\Theta(n \log n)$

Approach 3:

1. Build heap - $O(n)$
2. Run inner loop of heap sort 'k' times
   a. Exchange root with last element
   b. Then remove last element from heap
   c. sink(root) to fix heap

   $\Big\}\ O(\log n)$

RT: $O(n + k \log n)$

- if $k < \frac{n}{\log n}$

then RT $= O(n + \frac{n}{\log n} \times \log n)$
$= O(n + n)$
$= O(n)$

$n =$ 10 billion elements $- 10^{10}$

$\log_{10} n = 10$

if $k < \frac{10 \text{ billion}}{10} = 1$ billion

RT: $O(n)$

$\therefore$ For a 10 billion sized array, we can get top 1 billion elements in sorted order in $O(n)$